

מבני נתונים 1

יבש 1

מגישים:

רואן מחאגנה תז 206854051

שגיא רולנד תז 211548839

(1

(א

1. בהינתן $S_n = \{k | k \in \mathbb{N} \wedge 5^k \leq n\}$

נרצה לנתח את הפונקציה $f(n) = \sum_{k \in S_n} k$

נבחן את התנאי $5^k \leq n$. נלקח לוגריתם בסיס 5 משני האגפים ונקבל: $\log_5 n \geq k$

מכאן נובע ש- $S_n = \{1, 2, \dots, K\}$ כאשר $\lfloor \log_5 n \rfloor = K$

הסכום של $f(n)$ הוא סכום של סדרה חשבונית: $\frac{K(K+1)}{2} =$

$$K + \dots + 2 + 1 = f(n)$$

לערכים גדולים של n , מתקיים $K \sim \log_5 n$, ולכן:

$$f(n) \sim \frac{1}{2} K^2 \sim \frac{1}{2} (\log_5 n)^2$$

עכשיו נעבור ללוגריתמים טבעי לצורך כתיבת סדר גודל :

$$\log_5 n = \frac{\ln n}{\ln 5}$$

$$f(n) \sim \frac{1}{2} \left(\frac{\ln n}{\ln 5} \right)^2 = \Theta((\log n)^2): \text{לכן}$$

$$S_n = \{k | k \in \mathbb{N} \wedge 5^k \leq n\} \text{ 2. בהינתן}$$

$$g(n) = \sum_{k \in S_n} 4^k \text{ נרצה לנתח את הפונקציה}$$

נשים לב ש- S_n כולל את כל האינדקסים k הטבעיים כך

$$\text{ש- } 5^k \leq n \text{ כלומר, } S_n = \{1, 2, \dots, K\} \text{ כאשר}$$

$$\lfloor \log_5 n \rfloor = K$$

$$\text{ולכן: } g(n) = \sum_{k=1}^K 4^k$$

מדובר בסכום שך סדרה הנדסית מהצורה :

$$\sum_{k=1}^K 4^k = 4 + 4^2 + \dots + 4^K$$

שהוא סכום של סדרה הנדסית מהצורה :

$$\sum_{k=1}^K r^k = r \cdot \frac{r^K - 1}{r - 1}$$

עבור $r=4$ נקבל: $g(n) = 4 \cdot \frac{4^K - 1}{3} = \frac{4}{3}(4^K - 1)$

כאשר n גדול מאוד התרומה של (-1) זניחה ולכן:

$$g(n) \sim \frac{4}{3} \cdot 4^K$$

כעת נשתמש בזהות:

$$4^K = 4^{\log_5 n} = n^{\log_5 4}$$

ומקבלים ש- $g(n) = \Theta(n^{\log_5 4})$

(ב)

1. הטענה לא נכונה נפריך על ידי דוגמה נגדית

נבנה שני אלגוריתמים:

אלגוריתם א פועל ב $\Theta(\log(n))$

אלגוריתם ב פועל ב $\Theta(\log(n) - \log(\log(n)))$

נתן דוגמה נגדית, עבור $n=2$:

$$\log_2 2 = 1$$

$$\log_2(\log_2 2) = \log_2 1 = 0$$

אלגוריתם א: $\log_2 2 = 1$

אלגוריתם ב: $\log_2 2 - \log_2(\log_2 2) = 1 - 0 = 1$
כלומר קיבלנו עבור $n=2$ תוצאה שווה של שני האלגוריתמים.

2. הטענה נכונה

נבדוק האם קיים n כך ש :

$$\frac{\log(n)}{\log(\log(n))} < \log(n)$$

נחלק ב- $\log(n)$ בהנחה ש $n > 1$:

$$\frac{1}{\log(\log(n))} < 1$$

$$1 < \log(\log(n))$$

$$\log(n) > 10^1 = 10$$

$$n > 2^{10} = 1024$$

כלומר לכל $n > 1024$ זמן הריצה של האלגוריתם המשופר קטן מזה של המקורי.

לכן הטענה נכונה וכן קיים n (ולמעשה אינסוף ערכים עבור n גדול מספיק) שמקיים זאת.

(ג)

הטענה נכונה.

נניח בשלילה כי קיימות f, g כך שבה"כ $f = O(g)$ וגם $g \neq O(f)$

אזי לפי הגדרת O קיימים C ו- $n_1 > n$ כך שלכל $n > n_1$

$$F(n) \leq Cg(n)$$

וגם לכל $n > n_2$ קיים C כך ש $Cg(n) < F(n)$

אזי מתקיים כי $F(n) \leq Cg(n) < F(n)$

וזו סתירה ולכן $f \neq O(g)$ וגם $g \neq O(f)$

(ד

1. הוכחה

ההגדרה של $f(n) = \omega(g(n))$ היא :

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

נחשב :

$$\frac{h(n) \cdot f(n)}{h(n) \cdot g(n)} = \frac{f(n)}{g(n)}$$

ולכן :

$$\lim_{n \rightarrow \infty} \frac{h(n) \cdot f(n)}{h(n) \cdot g(n)} = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

כלומר,

$$h(n) \cdot f(n) = \omega(h(n) \cdot g(n))$$

2. הפרכה, נתן דוגמה נגדית

ניקח את הדוגמה הבאה :

$$g(n) = \frac{1}{n}$$

$$f(n) = n$$

$$h(n) = \frac{1}{n}$$

נחשב :

$$h(f(n)) = h(n) = \frac{1}{n}$$

$$h(g(n)) = h\left(\frac{1}{n}\right) = \frac{1}{\frac{1}{n}} = n$$

אז :

$$\frac{h(f(n))}{h(g(n))} = \frac{\frac{1}{n}}{n} = \frac{1}{n^2}$$

כעת נחשב את הגבול :

$$\lim_{n \rightarrow \infty} \frac{h(f(n))}{h(g(n))} = \lim_{n \rightarrow \infty} \frac{1}{n^2} = 0$$

לכן לא מתקיים $h(f(n)) = \omega(h(g(n)))$ בדוגמה הזו.
 (2) א) כיוון ש x ו y נשמרים כמערך נעבור בעזרת לולאה מקוננת שרצה על אברי המערכים. נשמור משתנה לתוצאה ובכל איטרציה נכפיל את האיבר הנוכחי ב x עם כל אחד מעברי המערך y וב shift של מכפלת האינדקסים כך נבטיח שמירה על יחידות מידה של ספרות כמו עשרות מאות ואלפים שהם 10^3 כלומר $\text{shift}(3)$.
 בסיום האיטרציה החיצונית הראשונה כפלנו את כל אברי המערך y ב איבר הראשון ב x .
 בסיום הריצה נקבל את הסכום המבוקש.
 הוכחת סיבוכיות:

פעולות לפני הלולאות: הגדרת משתנים זמניים $O(1)$
 פעולות אחרי הלולאות: החזרת ערך $O(1)$
 פעולות לולאה פנימית: כפל של 2 ספרות ו $shift()$ סה"כ
 $O(1)$

מספר האיטרציות בלולאה הפנימית $n=d(y)$ סך הכל
 $O(n)$

פעולות לולאה חיצונית: סה"כ $O(1)$
 מספר האיטרציות בלולאה החיצונית $n=d(x)$ סך הכל
 $O(n)$

ולכן סיבוכיות הזמן היא $O(n^2)$ כנדרש.

ב) הנוסחה הרקורסיבית $T(n) = 4T\left(\frac{n}{2}\right) + 7$
 הסבר: בכל שלב מחלקים את x ו y לשני מערכים
 שגודלם חצי מהקריאה הקודמת וקוראים לפונקציה
 הרקורסיבית לחישוב החלק הנוכחי סך הכל 4 פעמים
 כשכל פעם המערכים קטנים בחצי ומבצעים מספר קבוע
 של פעולות כפל.
 הוכחת סיבוכיות:

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{2}\right) + 7 = 4\left(4T\left(\frac{n}{2^2}\right) + 7\right) + 7 = \dots \\ &= 4^{i+1}T\left(\frac{n}{2^i}\right) + 4^i \cdot 7 + 7 \end{aligned}$$

$$\log(n) = i \Leftrightarrow 2^i = n \Leftrightarrow \frac{n}{2^i} = 1$$

$$4^{\log(n)+1} \cdot T(1) + 4^{\log(n)} \cdot 7 + 7 \Rightarrow O(n^{\log_2(4)})$$

ג) נציב את ההצעה החדשה בנוסחא המקורית:

$$\begin{aligned} x \cdot y &= x_h \cdot y_h \cdot 10^{2m} \\ &+ ((x_h + x_L)(y_h + y_L) - x_h \cdot y_h - x_L \cdot y_L) \\ &\cdot 10^m + x_L \cdot y_L \end{aligned}$$

השוני כעת הוא שבעזרת חישוב אלגברי איחדנו את הכפל ב- 10^m . בפנים ניתן כעת לקרוא לפונק' הרקורסיבית פעם אחת בלבד ולא פעמיים ולהגיע לאותה התוצאה עבור 3 קריאות בקורסיביות עם חצי מהמערך ו 8 פעולות כפל

$$T(n) = 3T\left(\frac{n}{2}\right) + 8$$

הוכחת סיבוכיות:

$$\begin{aligned} T(n) &= 3T\left(\frac{n}{2}\right) + 8 = 3\left(3T\left(\frac{n}{2^2}\right) + 8\right) + 8 = \dots \\ &= 3^{i+1}T\left(\frac{n}{2^i}\right) + 3^i \cdot 8 + 8 \end{aligned}$$

$$\log(n) = i \Leftrightarrow 2^i = n \Leftrightarrow \frac{n}{2^i} = 1$$

$$3^{\log(n)+1} \cdot T(1) + 3^{\log(n)} \cdot 8 + 8 \Rightarrow O(n^{\log_2(3)})$$

כלומר סיבוכיות הזמן לא השתנתה.

$$T(n) = (2k - 1)T\left(\frac{n}{k}\right) + 2^k n \quad \text{ד) מתן צודק. נמצא}$$

ביטוי ל $T(n)$ דרך שיטת המאסטר ואז נשתמש בהגדרת הגבול להוכיח $o()$

$$f(n) = 2^k n, \quad a = 2k - 1, \quad b = k, \quad \log_b(a) = \log_k(2k - 1)$$

$$n < n^{\log_k(2k-1)-\varepsilon} < n^{\log_k(2k)-\varepsilon} = n^{2-\varepsilon}$$

$$f(n) = O(n^{2-\varepsilon}) \leq 1$$

$$T(n) = \theta(n^{\log_k(2k-1)})$$

נשתמש בהגדרת הגבול להוכיח $o()$:

$$\lim_{n \rightarrow \infty} \frac{n^{\log_k(2k-1)}}{n^{1+\varepsilon}} = \lim_{n \rightarrow \infty} \frac{\log_k(2k-1) n^{\log_k(2k-1)-1}}{(1+\varepsilon)n^\varepsilon} = 0$$

כי עבור k מתאים $\log_k(2k-1) < 1 + \varepsilon$ ולכן

$$2k - 1 < k^{\varepsilon+1}$$

לכן הגבול שואף ל 0 ולפי ההגדרה $T(n) = o(n^{\varepsilon+1})$

כלומר מתן צודק.

$$T(n) = 2\sqrt{n}T(\sqrt{n}) + n(\log(n) - 1) \quad (ה)$$

$$\frac{T(n)}{n} = 2 \frac{T(\sqrt{n})}{\sqrt{n}} + \log(n) - 1$$

$$S(n) = \frac{T(n)}{n} \quad \text{נציב}$$

$$S(n) = 2S(\sqrt{n}) + \log(n) - 1$$

$$S(2^m) = 2S(2^{m/2}) + m - 1 \quad m = \log(n) \quad \text{נסמן}$$

$$U(m) = S(2^m) \quad \text{נציב}$$

$$U(m) = 2U\left(\frac{m}{2}\right) + m - 1$$

$$U(m) = \theta(m \log(m))$$

$$S(n) = S(2^m) = U(m) = \theta(m \log m)$$

$$= \theta(\log n * \log \log n)$$

$$T(n) = nS(n) = n * \theta(\log n * \log \log n) \\ = \theta(n * \log n * \log \log n)$$

נבדוק האם:

$$\lim_{n \rightarrow \infty} \frac{n \log_2(n) \log \log n}{n^{1+\varepsilon}} = 0$$

לא כי $\varepsilon > 1$ עבור $\log_2(n) \log \log n > \omega(n^{1+\varepsilon})$ ולכן $T(n) =$

3) אציג אלגוריתם למימוש הפעולות עבור מבנה נתונים של מערך

$init(n, t)$: אתחול מערך ל 0 בסיבוכיות $O(1)$ בזמן וב $O(n)$ במקום:

נייצר 3 מערכים בגודל n סהכ $O(n)$ סיבוכיות מקום נוסף ועוד $O(1)$ ליצירת משתנה t .

מערכים אלו יהיו מלאים בערכי זבל נסמן אותם ב A, B, C כאשר A - יהיה מערך הערכים, B -מערך קיצורי הדרך, C - מערך השומר אינדקסים עם ערך ועם מצביע top לערך האחרון שהוכנס ל C .

תיאור האלגוריתם:

נכניס ערך ל $A[i]$ עם set ב $O(1)$.

נגדיר פונק' לזיהוי אינדקס מאותחל: אם הערך $top >$
 $B[i]$ וגם $B[i] \Rightarrow 0$ וגם במערך האינדקסים עם ערך C
הערך של C במקום $B[i]$ שווה ל i .
כלומר אינדקס מופיע ב C בין 0 ל top אמ"מ הוא
מאותחל.
במערך A יהיה מצביע בין האינדקסים המקבילים ב B .
כאשר נכניס ערך ל A באינדקס i ב $O(1)$ עם set נבדוק אם
הערך לא מאותחל ואם הוא לא נבצע: $store$
 $i = C[top]$ ולכן לפי הטענות מעל $top = B[i]$.
נגדיל את top ב 1 ואז נבצע את ההשמה $A[i] = e$
סהכ סיבוביות זמן $O(1)$. וסיבוכיות מקום $O(n)$.
 $Reset()$: נשתמש אותו אלגוריתם אתחול מערך שהצגנו
ב $init(n, t)$ ולפי ההוכחה סיבוכיות הזמן היא $O(1)$.
 $write(i, e)$: נשתמש באותו אלגוריתם של $store$ בסיבוכיות
זמן $O(1)$.
 $Read(i)$: נחזיק מערך נוסף באורך n מאותחל ל 0 . $O(1)$
בזמן $O(n)$ זיכרון ונשנה את מבנה הנתונים כך שב
 $write$ של הערכים וב $read$ במערך הנוסף נשים $+1$
באינדקס ה i עבור $write$ ו- 0 עבור $read$.
ונשווה את הערך ב i ל t אם הוא גדול מ t נחזיר 0 ,
אחרת את הערך של המערך המקורי ב i .

שאלה 4 :
(א

```
class MyArray {  
private:  
    int* arr;  
    int n;  
  
public:  
    MyArray() : arr(nullptr), n(0) {}  
  
    void init(int size) {
```

```
n = size;
arr = new int[n];
for (int i = 0; i < n; ++i)
    arr[i] = 0;
}
```

```
void write(int index, int val) {
שימוש בפוינטר    *(arr + index) = val; //
}
```

```
int sum(int a, int b) {
    int total = 0;
    int i = a;
    while (i <= b) {
        total += arr[i];
        ++i;
    }
    return total;
}
```

```
~MyArray() {
    delete[] arr;
}
```

};

הקוד מגדיר מחלקה (class) בשם MyArray שתפקידה לנהל מערך של מספרים שלמים (int). המערך אינו קבוע מראש, אלא מוקצה בזיכרון הדינאמי (heap) בזמן שהמשתמש בוחר את הגודל. המחלקה כוללת יכולות בסיסיות: ליצור את המערך, לכתוב לתוכו, לחשב סכום של תאים, ולשחרר את הזיכרון כשלא צריך אותו. יש שני משתנים פרטיים:

- משתנה אחד הוא מצביע (כלומר כתובת בזיכרון) שמצביע על תחילת המערך.
- השני הוא מספר שלם שמייצג את הגודל של המערך, כלומר כמה תאים יש בו.

בהתחלה, כשיוצרים אובייקט מהמחלקה, המצביע ריק (לא מצביע לשום מקום), והגודל אפס. בשלב הזה, אין מערך עדיין.

המחלקה כוללת פונקציה שמקבלת מספר שלם ומקצה בהתאם לכך מערך בגודל המבוקש בזיכרון הדינאמי. לאחר ההקצאה, היא גם דואגת לאתחל את כל תאי המערך לאפסים – כדי שהמשתמש לא יקבל ערכים אקראיים. הפונקציה גם שומרת את הגודל שהוזן כדי לזכור כמה תאים יש במערך שנוצר.

יש פונקציה נוספת שמקבלת שני ערכים: אינדקס (מיקום בתוך המערך) וערך. היא שמה את הערך הזה במקום המתאים במערך. השימוש נעשה דרך מצביעים – כלומר, במקום לגשת ישירות לתא על פי הסוגריים המרובעים, המחלקה משתמשת בחשבון כתובות בזיכרון כדי להגיע למיקום הנכון ולשנות אותו.

המחלקה כוללת פונקציה שמקבלת שני אינדקסים – התחלה וסוף – ומחזירה את הסכום של כל הערכים שנמצאים בתאים בין שני האינדקסים (כולל שניהם). היא עושה זאת על ידי לולאה שמתחילה מהאינדקס הראשון וממשיכה עד האינדקס האחרון, מוסיפה כל ערך למשתנה ביניים, ולבסוף מחזירה את הסכום הכולל.

המחלקה כוללת דסטרוקטור – כלומר פונקציה מיוחדת שמופעלת באופן אוטומטי כשמסיימים להשתמש באובייקט. הפונקציה הזאת משחררת את הזיכרון של המערך מה־heap, כדי למנוע מצב של דליפת זיכרון (מצב שבו זיכרון שהוקצה לא משתחרר, למרות שלא משתמשים בו יותר).

(ב)

```
class MyArray {  
private:
```

```
int* arr;  
int* prefix;  
int n;
```

public:

```
MyArray(int size) {  
    n = size;  
    arr = new int[n]();  
    prefix = new int[n]();  
    // Both arrays initialized to 0  
}
```

```
void write(int index, int val) {  
    int diff = val - arr[index];  
    arr[index] = val;  
    // Update prefix sums from index to end  
    for (int i = index; i < n; ++i) {  
        prefix[i] += diff;  
    }  
}
```

```
int sum(int a, int b) {  
    if (a == 0) return prefix[b];
```

```

    else return prefix[b] - prefix[a - 1];
}

~MyArray() {
    delete[] arr;
    delete[] prefix;
}
};

```

המחלקה MyArray יוצרת מערך רגיל של מספרים שלמים, ובנוסף מחזיקה מערך נוסף שמכיל את הסכומים המצטברים של המערך הראשי. הרעיון הוא שברגע שמעדכנים את אחד הערכים, המערך השני מתעדכן כך שאפשר לחשב סכומים בין כל שני אינדקסים ביעילות – בזמן קבוע או לינארי קטן.

יש שלושה משתנים עיקריים:

1. מצביע למערך הערכים עצמם (הנתונים הגולמיים).
2. מצביע למערך סכומים מצטברים – בכל תא במערך הזה מאוחסן סכום כל הערכים מהתחלה ועד אותו אינדקס.

3. מספר שלם שמייצג את גודל המערך.
- כלומר, יש כאן שני מערכים בגודל זהה:

- הראשון מאחסן את הנתונים עצמם.
- השני מאחסן את סכומם, מצטבר עד כל מיקום.

ברגע שיוצרים מופע של המחלקה, המשתמש חייב להעביר את הגודל הרצוי. בשלב הזה מוקצים שני מערכים בזיכרון: האחד עבור הערכים, והשני עבור הסכומים. שניהם מאותחלים מראש לאפסים, כך שאין צורך לאתחל ידנית.

כאשר המשתמש כותב ערך חדש לתא מסוים, מתבצע חישוב של ההפרש בין הערך החדש לבין מה שהיה קודם באותו תא. ההפרש הזה מתווסף לכל התאים במערך הסכומים מהאינדקס הזה ועד סוף המערך. המשמעות היא שמערך הסכומים תמיד נשאר מעודכן בהתאם לערכים שנמצאים כרגע במערך הראשי.

כדי לחשב את הסכום בין שני אינדקסים במערך, ניגשים למערך הסכומים ובודקים:

- אם הטווח מתחיל מההתחלה (כלומר מהאינדקס 0), פשוט לוקחים את הסכום המצטבר של האינדקס הסופי.

- אם הטווח מתחיל מאמצע המערך, מחשבים את ההפרש בין הסכום עד הסוף לבין הסכום עד אחד לפני ההתחלה.

כך, במקום לעבור על כל התאים בכל פעם (כמו בקוד הקודם), כאן ניתן לחשב סכום של טווח בזמן קבוע. $O(1)$ כאשר האובייקט נהרס (למשל, כשהתוכנית מסתיימת או כשמשתנה מקומי יוצא מהקשר), משתחררים שני המערכים שהוקצו בזיכרון הדינאמי. זה מונע דליפת זיכרון.

ההבדל בין סעיף א וב :
הוא מאפשר **עדכון של ערכים ביעילות**, תוך שמירה על התאמה במערך הסכומים.
הוא מאפשר **חישוב סכומים בטווחים בצורה מהירה מאוד**.

(ג)

```
#include <cmath>
```

```
class MyArray {  
private:  
    int* values;
```

```
int* blockSum;  
int* blockPrefixSum;  
int n, blockSize, numBlocks;
```

public:

```
MyArray(int size) {  
    n = size;  
    blockSize = (int)std::sqrt(n) + 1;  
    numBlocks = (n + blockSize - 1) / blockSize;  
    values = new int[n]();  
    blockSum = new int[numBlocks]();  
    blockPrefixSum = new int[numBlocks]();  
    // All arrays initialized to 0  
}
```

```
void write(int index, int val) {  
    int block = index / blockSize;  
    int diff = val - values[index];  
    values[index] = val;  
    blockSum[block] += diff;
```

```
    // Update prefix sums of blocks from this block  
    onward
```

```

    for (int i = block; i < numBlocks; ++i) {
        blockPrefixSum[i] += diff;
    }
}

```

// Sum from the beginning to the end of block b
(inclusive)

```

int sumOfBlocks(int b) {
    return blockPrefixSum[b];
}

```

// Sum of any range [a, b]

```

int sum(int a, int b) {

```

// For arbitrary ranges, use prefix sum on the
whole array (see solution B)

// For block-aligned ranges:

```

    int startBlock = a / blockSize;

```

```

    int endBlock = b / blockSize;

```

```

    if (a % blockSize == 0 && b == (endBlock + 1) *
        blockSize - 1 && b < n) {

```

// Full blocks only

```

        if (startBlock == 0)

```

```

        return blockPrefixSum[endBlock];
    else
        return blockPrefixSum[endBlock] -
blockPrefixSum[startBlock - 1];
    }

```

// Otherwise, fall back to direct sum for partial blocks:

```

    int sum = 0;
    for (int i = a; i <= b; ++i)
        sum += values[i];
    return sum;
}

```

```

~MyArray() {
    delete[] values;
    delete[] blockSum;
    delete[] blockPrefixSum;
}
};

```

הקוד הזה הוא שדרוג נוסף ומתקדם יותר של המחלקה

Square Root שמיישם רעיון שנקרא **Decomposition** – פירוק למקטעים בגודל שורש גודל

המעריך, כדי לאזן בין פעולות עדכון לבין שאילות של סכום בטווח. זהו אלגוריתם שמשפר ביצועים כאשר יש הרבה קריאות לסכומים בטווחים משתנים.

1. **מעריך ערכים רגיל** – מאחסן את המספרים עצמם שהמשתמש מזין.

2. **מעריך סכומים של בלוקים** – מחלק את המעריך לקבוצות (בלוקים), ובכל תא במעריך זה נשמר הסכום של כל הערכים בבלוק מסוים.

3. **מעריך סכומים מצטברים של בלוקים** – כל תא במעריך זה שומר את סכום כל הבלוקים עד אותו בלוק כולל. בנוסף, נשמרים:

- גודל המעריך הכולל. (n)
- גודל של כל בלוק (בערך שורש של n).
- מספר הבלוקים שנדרש כדי לכסות את כל המעריך.

כאשר רוצים לחשב סכום של חלק מהמעריך (למשל, בין אינדקס a ל b), אפשר לעיתים להשתמש רק בסכומים שהצטברו בבלוקים – במקום לעבור איבר-איבר. זה מאפשר קפיצה משמעותית בביצועים בשאילות שמכסות טווחים רחבים.

השיטה מבוססת על איזון:

- פעולת עדכון דורשת לעדכן בלוק אחד + סכומי בלוקים (כלומר פעולה שגדלה בערך לפי $\sqrt{n}$)

- פעולת סכום בטווח רץ בצורה מהירה, במיוחד אם הטווח חוצה בלוקים שלמים.

כאשר יוצרים מופע חדש של המחלקה, המשתמש מוסר את גודל המערך הרצוי. הקוד מחשב את גודל הבלוק (שורש של n מעוגל למעלה) ואת מספר הבלוקים הדרושים. לאחר מכן מוקצים שלושה מערכים: אחד לערכים עצמם, אחד לסכומי בלוקים, ואחד לסכומים המצטברים של הבלוקים. כולם מאותחלים לאפס.

כאשר מזינים ערך חדש באינדקס מסוים:

- הקוד מחשב לאיזה בלוק שייך האינדקס.
- מחשב את ההפרש בין הערך הישן לחדש.
- מעדכן את הערך החדש במערך הראשי.
- מוסיף את ההפרש לסכום של הבלוק שאליו שייך האינדקס.
- מעדכן את סכומי הבלוקים המצטברים – בכל הבלוקים מאותו בלוק ואילך (כי הם כוללים את הבלוק הזה).

כאן נמצאת העוצמה של השיטה:

1. אם הטווח המבוקש מתחיל בתחילת בלוק ונגמר בסוף בלוק שלם, אפשר להשתמש אך ורק במערך הסכומים המצטברים של בלוקים. זו פעולה מהירה במיוחד.

2. אם הטווח לא מתאים בדיוק לגבולות של בלוקים –
כלומר, אם מתחילים או מסיימים באמצע בלוק – הקוד
עובר ידנית על כל איבר בטווח ומחשב את הסכום
באיטיות יחסית.

כך מתקבל איזון: חלק מהשאלות ייענו בזמן כמעט קבוע,
וחלק עדיין בזמן לינארי (אבל קצר יותר).

בסיום השימוש, הדסטרקטור משחרר את כל הזיכרון
שהוקצה באופן דינמי – שלושת המערכים. זה חשוב כדי
למנוע דליפות זיכרון.